Design and Analysis of Algorithms Project

Anurag Ramachandran[1] Aryan Avinash Komarla[2]

[1]Roll Number: bmat2310
[2]Roll Number: bmat2411

Contents

1 Introduction to the Problem Statement and the Algorithm **2**
- 1.1 Problem Statement . . . 2
- 1.2 Explanation of the Algorithm . . . 3
- 1.3 Analysis of the Time Complexity of the Algorithm . . . 4

2 Proof of Correctness and Implementation of the Algorithm in Python **5**
- 2.1 Proof of Correctness of the Algorithm . . . 5
- 2.2 Implementation of the Algorithm in Python . . . 6
  - 2.2.1 Python Implementation . . . 6
- 2.3 Examples of Implementation . . . 8
  - 2.3.1 Example 1: A Path with Three Vertices . . . 8
  - 2.3.2 Example 2: A Root with Two Children with Each Having One Child . . . 8
  - 2.3.3 Example 3: A Path with Five Vertices . . . 8
- 2.4 Execution of the Examples Using PythonImmediate . . . 9
- 2.5 Acknowledgments . . . 9

Chapter 1

Introduction to the Problem Statement and the Algorithm

1.1 Problem Statement

In organization ABC, the employees have an underlying hierarchy, where the CEO is at the top and every other employee has exactly one manager. An employee can be the manager of one or more employees. Any employee is familiar with the work of her manager and the work of her immediate subordinates.

Problem 1.1.1. *ABC decides to constitute a communication team. A communication team consists of employees such that, for every employee who is not in the team, there is a person in the communication team who is familiar with her work. Further, the management decides that if the team consists of a particular employee, then her manager cannot be present in the communication team, and for every employee who is not in the team, there is exactly one person in the communication team who is familiar with her work. Give an algorithm that selects such a communication team of the smallest possible size, or returns that no such team can exist.*

We provide a natural graph theoretic reformulation of the problem. We use this to explain the algorithm and to prove its correctness. We construct a directed graph $G = (V, E)$ with vertex set V representing the employees. We draw a directed edge from vertex u to vertex v if the employee corresponding to u is the manager of the employee corresponding to v.

The hierarchy mentioned in the question is assumed to imply that the graph is connected when viewed as an undirected graph, that is, one can trace back an employee's graph ancestors to the CEO after removing the directions from each edge. Furthermore, let the number of employees in the company be n.

The CEO represents the root of the graph. Note that the graph G contains no undirected cycles. This is because the graph contains $n-1$ edges corresponding to the $n-1$ employees apart from the CEO, each of whom has exactly one manager. A connected and undirected graph with n vertices and $n-1$ edges must be a tree, which proves our claim. We restate the problem using graph theory.

Problem 1.1.2. *Let T be a rooted directed tree. Find an algorithm to determine, if it exists, the cardinality of the smallest subset $S \subseteq V(T)$ such that*

- *S forms an independent set in T, which means that no two vertices u and v in S have a directed edge between them.*
- *Every vertex $v \in V(T) \setminus S$ has exactly one neighbour $u \in S$ such that there is a directed edge from u to v or v to u.*

We now describe a dynamic programming algorithm that will determine the smallest such subset of the vertex set, if it exists.

1.2 Explanation of the Algorithm

We begin by introducing the necessary parameters. For each vertex v, let T_v be the induced subgraph of T comprising the vertex v and all its directed descendants (that is, the vertices u for which there is a directed path from v to u), with v as the root. We often break the problem down into subproblems involving these subgraphs T_v for $v \in V(T)$.

Define $A_v[x, y]$ as the smallest number of team members in T_v among all valid teams chosen within the rooted tree T_v, where $x \in \{0, 1\}$ records whether the parent of v is in the team, and $y \in \{0, 1\}$ records whether v is in the team. Inclusion is represented by 1 and exclusion is represented by 0.

Let $C(v)$ denote the immediate subordinates of v in the directed rooted tree. We define the value of $A_v[x, y]$ recursively along each directed path, given by the piecewise equation

$$A_v[x,y] = \begin{cases} +\infty & \text{if } (x,y) = (1,1), \\ +\infty & \text{if } (x,y) = (0,0) \text{ and } v \text{ is a leaf}, \\ 1 + \displaystyle\sum_{u \in C(v)} A_u[1,0] & \text{if } (x,y) = (0,1), \\ \displaystyle\sum_{u \in C(v)} A_u[0,0] & \text{if } (x,y) = (1,0), \\ \displaystyle\min_{w \in C(v)} \left[A_w[0,1] + \sum_{\substack{z \in C(v) \\ z \neq w}} A_z[0,0] \right] & \text{if } (x,y) = (0,0) \text{ and } v \text{ is not a leaf}. \end{cases}$$

Here, the convention chosen is that an empty sum is equal to 0, and the smallest of an empty set of numbers is taken to be $+\infty$. The reasoning behind these choices is given below.

1. We want to override assignments in which both a vertex and its parent are selected, or neither is selected when the vertex is a leaf. These form invalid assignments.

2. If a vertex v is selected but its parent is not, the configuration forbids us from choosing its children. In this case, the state for each child $u \in C(v)$ being $(x, y) = (1, 0)$ corresponds to v being selected and u being excluded. This means that we must sum over all the remaining branches.

3. A similar logic holds when the vertex v is not selected but its parent is. Note that for a unique neighbour to be selected in the team, none of the children can be selected either. This results in the state $(0, 0)$ for each child, corresponding to neither the child nor its parent being selected. We sum over all such children.

4. Finally, when the state for v is $(0, 0)$ and v has children, we can choose exactly one of the children of v to be in the team. We take the smallest of

$$A_w[0,1] + \sum_{\substack{z \in C(v) \\ z \neq w}} A_z[0,0],$$

over all $w \in C(v)$. The state $(0, 1)$ for w means that w is selected, while the state $(0, 0)$ for each other child $z \neq w$ means that z is not selected.

The algorithm is to evaluate $\min(A_{CEO}(0,0), A_{CEO}(0,1))$, where the vertex CEO is the root of the full tree. We will eventually show that the algorithm generates a valid team of the smallest possible size. We are going to analyze the above algorithm.

1.3 Analysis of the Time Complexity of the Algorithm

We let n denote the number of vertices in the directed tree T. The complexity of creating the child lists and passing through each employee and each manager edge once is $O(n)$, considering that the number of edges is linear in n.

The amount of work done at a vertex v is at most a linear in terms of the size of $C(v)$. These sums in any of the four cases, if the sum exists, can be constructed in $(O(|C(v)|))$ time, after which each candidate corresponding to a child can be evaluated in constant time. Thus, computing all four states at v takes $(O(1 + |C(v)|))$ time, which is still $O(n)$.

Finally, setting up the states of the descendants contributes another $O(n)$ term. All in all, the time complexity of running the algorithm is $O(n)$. We present the pseudocode for the algorithm. We represent the root by r for our convenience.

Algorithm 1 UniqueDomIndpSet

Require: A rooted directed tree T with root r
Ensure: A smallest valid set S, or report that no such set exists

Traverse T in from bottom to top order
for each vertex v in bottom to top order **do**
    $A_v[1,1] \leftarrow +\infty$
    $A_v[0,1] \leftarrow 1 + \sum_{u \in C(v)} A_u[1,0]$
    $A_v[1,0] \leftarrow \sum_{u \in C(v)} A_u[0,0]$
    if v is a leaf **then**
        $A_v[0,0] \leftarrow +\infty$
    else
        $A_v[0,0] \leftarrow \min_{w \in C(v)} \left(A_w[0,1] + \sum_{\substack{z \in C(v) \\ z \neq w}} A_z[0,0] \right)$
        Store a vertex w attaining the smallest
    end if
end for
$m \leftarrow \min\{A_r[0,0], A_r[0,1]\}$
if $m = +\infty$ **then**
    Report that no such set exists
else
    Recover S by backtracking from the chosen root state
    return S
end if

Chapter 2

Proof of Correctness and Implementation of the Algorithm in Python

2.1 Proof of Correctness of the Algorithm

We prove that the algorithm returns a valid independent set of the smallest size, when such a set exists and reports when no such set exists.

Recall that T_v denotes the subtree rooted at v, and $A_v[x, y]$ denotes the smallest number of selected vertices in T_v, where x indicates whether the parent of v is selected and y indicates whether v itself is selected. We prove the correctness of the recurrence by inducting on the height of the subtree rooted at v.

For the base case, suppose that v is a leaf. If v is selected, the corresponding state is $A_v[0, 1] = 1$. If its parent is selected and v is not selected, then $A_v[1, 0] = 0$. However, if neither v nor its parent is selected, the vertex v has no selected neighbour, and hence $A_v[0, 0] = +\infty$. The state $A_v[1, 1]$ is also invalid because it violates the independence condition. Thus, we observe that the recurrence gives the correct values for a leaf.

For the induction step, suppose that the recurrence correctly computes the smallest values for the subtrees rooted at all children of a vertex v. We consider the possible states of v case by case.

If v is selected, its parent and all its children must be excluded from the team. The smallest size in this case is

$$A_v[0, 1] = 1 + \sum_{u \in C(v)} A_u[1, 0]$$

If v is not selected but its parent is selected, then no child of v can be selected, since v already has a selected neighbour. Hence, we get that

$$A_v[1, 0] = \sum_{u \in C(v)} A_u[0, 0]$$

Finally, suppose that neither v nor its parent is selected. Since every vertex outside the set must have exactly one selected neighbour, precisely one child of v must be selected. The algorithm considers every possible choice of this child and takes the smallest, given by

$$A_v[0, 0] = \min_{w \in C(v)} \left[A_w[0, 1] + \sum_{\substack{z \in C(v) \\ z \neq w}} A_z[0, 0] \right]$$

The remaining state given by $A_v[1, 1]$ is invalid by the independence of the dominating set. By the induction hypothesis, all the states of the children are computed correctly. Consequently, the recurrence

computes the correct smallest for every state of v.

By induction, all dynamic programming states are computed correctly. At the root r, the parent is absent and so the smallest size of a valid set is

$$\min\left(A_r[0,0], A_r[0,1]\right)$$

If both values are infinite, no valid set exists. Otherwise, the choices recorded during the computation allow us to reconstruct a valid set attaining the smallest.

The algorithm returns a smallest valid set whenever one exists and correctly reports when such a set does not exist.

2.2 Implementation of the Algorithm in Python

We now implement the algorithm in Python. The tree is represented in the numerical order of the appearance of vertices in order of their parent's number.

2.2.1 Python Implementation

The implementation is as follows.

```
from math import inf


def udip(children, root):
    order = []
    stack = [(root, False)]

    while stack:
        v, visited = stack.pop()

        if visited:
            order.append(v)
        else:
            stack.append((v, True))
            for u in children[v]:
                stack.append((u, False))

    A = {}
    choice = {}

    for v in order:
        A[v] = {
            (1, 1): inf,
            (0, 1): 1 + sum(A[u][(1, 0)] for u in children[v]),
            (1, 0): sum(A[u][(0, 0)] for u in children[v])
        }

        best = inf
        best_child = None

        prefix = [0] * (len(children[v]) + 1)
```

```
    for i, u in enumerate(children[v]):
        prefix[i + 1] = prefix[i] + A[u][(0, 0)]

    suffix = [0] * (len(children[v]) + 1)
    for i in range(len(children[v]) - 1, -1, -1):
        u = children[v][i]
        suffix[i] = A[u][(0, 0)] + suffix[i + 1]

    for i, w in enumerate(children[v]):
        value = (
            A[w][(0, 1)]
            + prefix[i]
            + suffix[i + 1]
        )

        if value < best:
            best = value
            best_child = w

    A[v][(0, 0)] = best
    choice[(v, 0, 0)] = best_child

root_state = min(
    [(0, 0), (0, 1)],
    key=lambda state: A[root][state]
)

if A[root][root_state] == inf:
    return None

S = set()
stack = [(root, 0, root_state[1])]

while stack:
    v, x, y = stack.pop()

    if y:
        S.add(v)
        stack.extend((u, 1, 0) for u in children[v])

    elif x:
        stack.extend((u, 0, 0) for u in children[v])

    else:
        w = choice[(v, 0, 0)]

        for u in children[v]:
            stack.append((u, 0, int(u == w)))

return S
```

2.3 Examples of Implementation

We consider three examples to illustrate the implementation. In each example, the tree is represented by the numerical order of the children.

2.3.1 Example 1: A Path with Three Vertices

Consider the directed path $0 \to 1 \to 2$, rooted at 0. The set $\{1\}$ is a smallest unique independent dominating set, as vertex 0 is dominated by its child 1 and vertex 2 is dominated by its parent 1.

```
children = {
    0: [1],
    1: [2],
    2: []
}

print(udip(children, 0))
```

```
{1}
```

2.3.2 Example 2: A Root with Two Children with Each Having One Child

Consider a tree rooted at 0 with the children 1 and 2, where 1 has a child 3 and 2 has a child 4. The set $\{1, 4\}$ is a unique independent dominating set of smallest size. This is because the vertices 0 and 3 are dominated by 1, while vertex 2 is dominated by 4.

```
children = {
    0: [1, 2],
    1: [3],
    2: [4],
    3: [],
    4: []
}

print(udip(children, 0))
```

```
{1, 4}
```

2.3.3 Example 3: A Path with Five Vertices

Consider the directed path

$$0 \to 1 \to 2 \to 3 \to 4,$$

rooted at 0. The set $\{1, 4\}$ is a unique independent dominating set of smallest size. The vertices 0 and 2 are dominated by 1, while the vertex 3 is dominated by 4.

```
children = {
    0: [1],
    1: [2],
    2: [3],
    3: [4],
    4: []
}

print(udip(children, 0))
```

```
{1, 4}
```

2.4 Execution of the Examples Using PythonImmediate

The following code executes the implementation and all three examples during compilation. It uses the same recurrence as the implementation above.

Remark 2.4.1. *We are using the pythonimmediate package, which renders the Python code in LaTeX. Integration of the package is mentioned on the GitHub page. The code shown below is the compiled result of the implementation.*

```
Example 1: {1}
Example 2: {1, 4}
Example 3: {1, 4}
```

2.5 Acknowledgments

The authors would like to declare that generative LLM models were used to integrate the pythonimmediate package into the LaTeX file, to generate the pseudocode according to the syntax of the algorithms package. Furthermore, the python code wrapped in verbatim and the pythonimmediate packages were generated by LLMs, as neither of the authors are proficient with Python. The GitHub page for the project is hosted here.